

MINI PROJECT REPORT

Institute Name: BESANT TECHNOLOGIES

Project Name: ROCK, PAPER, SCISSORS GAME USING PYTHON

Submitted To: Smt. GOWTHAMI S N

Submitted By: Shri. MANUEL MARIA BASTIN S

Date of Submission: 30.05.2026

Github_link: https://github.com/ManuelMariaBastin/Python_projects

ACKNOWLEDGEMENT

First of all, I would like to thank my Wife for her unwavering support and motivation to improve my skills in programming and helping in smooth finishing of project.

I would like to express my sincere gratitude to my guide for her valuable support and encouragement for the completion of this mini project. Their guidance helped me understand the various concepts of Python programming with weekly assignments and questions for applying them practically in building project. This mini project helped me improve my programming skills, logical thinking, and understanding of basic Python concepts .

I also thank institution for providing the opportunity and resources to work on this project.

DECLARATION

I hereby declare that the mini project titled "**Rock Paper Scissors Game Using Python**" submitted in partial fulfillment of the requirements for the course is **my original work**. The concepts, design, coding, testing, and implementation of the Mini- project have been completed by me as per instructions received.

INDEX

- 1. Cover Page**
- 2. Acknowledgement**
- 3. Declaration**
- 4. Index**
- 5. Introduction to Python**
- 6. Project Description**
- 7. Concepts Used in the Project**
 - **Variables**
 - **Conditional Statements**
 - **Iterative Statements**
 - **Loop Control Statements**
 - **Importing Random Module**
 - **Input, Print statement**
- 8. Source Code**
- 9. Explanation of Source Code**
- 10. Output**
- 11. Conclusion**
- 12. References**

INTRODUCTION TO PYTHON

Python is a **high-level, interpreted, and general-purpose** programming language. It is one of the most popular programming languages in the world. Python language was created by **Guido Van Rossum** in 1991 and its third version for public use. He named "Python" after a famous Monty "Python's Flying Circus" show.

Python is especially suitable for beginners because of **readability its simple syntax** (English like) is clear and extensible (support to other languages), **dynamically typed** (No explicit data type declaration), and platform independent language (works on different platform by PVM). Another reason for Python's popularity is its **portability** and it is called **cross platform language** can run on different operating systems such as Windows, Linux, and macOS with very little or no modification. It is also **open-source** and free to use, which makes it accessible to students, teachers, professionals, and organizations.

Python programs are executed line by line using an interpreter. This makes debugging easier because errors can be identified quickly. Generally considered as a **interpreted language** but internally it is compiled first (byte code) i.e., generally programs is written in editor which is compiled converted into byte code (low level)(pyc files) and then Python virtual machine (PVM interpreter) interprets line by line (machine code) giving desired result at instance.

Python also provides a **large standard library**, which contains built-in modules and functions for many tasks such as random number generation, file handling, mathematics, date and time operations, and more. This reduces the need to write everything from scratch and complex problems can be solved using inbuilt **modules (files), packages (folders) and libraries (directory)**.

One of the main advantages of Python is that it allows programmers to write fewer lines of code compared to many other programming languages. Also, it supports **multiple programming** paradigms such as procedural programming, object-oriented programming, and functional programming. This makes the development process faster and easier.

Because of this flexibility, developers can use Python and its **modules, libraries and its different flavors** in a variety of applications like Web development (backend using DJANGO (huge data), flask, bottles (micro), pyramids, web2py, cherrypy), Desktop GUI (Tkinter, Kivy), Data science & artificial intelligence (Numpy (data calculation), Pandas (data

manipulation), Matplotlib (Visualisation), seaborn (high level visualisation), Games (PyGames) , Software engineering, Audio & Video App (Gstreamer)..etc

Python software can be downloaded from the official website (<https://www.python.org/>) and depending upon the OS platforms, suitable python versions can be downloaded and after downloading following steps will get a Python installed and also environment path variables can be set so that it can be run from any files/directory.

In windows environment, We can also check whether Python is running, by opening command prompt and typing Python and it will show the current versions, i.e., **3.14.4** in which **3 indicates major level, 14 indicates minor level (14 new added in python)** and **4 indicates minor level (4 errors are corrected)**.

Most Python implementations (including **CPython (Python written in C)**, **PyPy (written in python)**) include a read-eval-print loop (REPL); this permits the environment to function as a command line interpreter, with which users enter statements sequentially and receive results immediately. CPython is bundled with an integrated development environment (IDE) called **IDLE** which is oriented toward beginners.

Other shells, including IDLE and IPython, add additional capabilities such as improved auto-completion, session-state retention, and syntax highlighting and Standard desktop IDEs include **PyCharm, Spyder, and Visual Studio Code** here are web browser-based IDEs, such as the following environments: **Jupyter Notebooks**, an open-source interactive computing platform, **PythonAnywhere**, a browser-based IDE and hosting environment; and **Canopy**, a commercial IDE from Enthought that emphasizes scientific computing.

.

PROJECT DESCRIPTION

In this mini project, the game is developed where the user plays against the computer. The project demonstrates the use of programming concepts such as variables, conditional statements, iterative statements, random choice generation, and user interaction.

The main objectives of the Rock Paper Scissors mini project are:

- To develop a simple interactive game using programming concepts
- To understand the use of conditional statements, iterative statements,. Etc
- To implement random choice generation using random module.

The mini project named **ROCK, PAPER, SCISSORS GAME** is developed using following like Programming Language: Python (Python 3.9.6) and IDE: Visual Studio Code (1.122.1 (Universal))

The game works by allowing the user player to choose one option among Rock, Paper, or Scissors using input function . The computer also randomly selects one option using random module. Based on predefined game rules like following,

Player 1	Player 2	Output
Rock	Scissors	Rock wins
Paper	Rock	Paper wins
Scissor	Paper	Scissor wins
Rock	Rock	Tie
Paper	Paper	Tie
Scissor	Scissor	Tie

Table 1: Rock Paper Scissors Game Logic

Likewise program compares both choices, tracks rounds, scores and declares the winner. The game can be played repeatedly, and scores are maintained throughout the session and it display the rounds, scores and winners.

CONCEPTS USED IN THE PROJECT

1. Variables

A variable is a container used to store data values in a program. Variables help store information that can be used and modified during program execution. In Python, variables do not need an explicit declaration of data type and the type of the variable is determined automatically based on the value assigned to it.

Rules for Naming Variables

The following rules should be followed while naming variables in Python:

1. Variable names must start with letter (**A–Z or a–z**) or an **underscore (_)**
2. Variable names **cannot start** with a number.
3. Variable names can contain **letters, Numbers, Underscores**
4. Spaces are not allowed in variable names.
5. Python keywords cannot be used as variable names.
6. Variable names are case-sensitive. i.e., score and Score are different variables.

The standard syntax is **Var_name = data_type** .

For example, various data types are

```
Score = 10           #Integer data type with score variable,  
Name = "Ram"         #String data type with name variable,  
Avg = 98.95          #Float data type for decimal representation  
Check = True         #Bool data type to verify true or false  
Data = None          #None data type to specific no data
```

2. Conditional Statements

Conditional statements are used to make decisions in a program. That is, it is used to control the flow of execution based on specific conditions, allowing a program to make decisions.

In Python, the conditional statements are **if statements, if-else statements and if-elif-else statements**. These statements allow the program to check conditions using **comparison operators** and **logical operators** to evaluate conditions and execute different blocks of code depending on whether the condition is **true** or **false**.

In this "**if-elif-else**" Statement blocks is used when multiple conditions need to be checked and

Syntax ,

```
if condition1:
    statement1
elif condition2:
    statement2
else:
    statement3
```

Comparison Operators	Logical Operators
== ---> Equal to,	and --> Both conditions must be true,
!= ---> Not equal to,	or --> At least one condition must be true
> ---> Greater than,	not --> Reverses the condition
< ---> Less than,	
>= --> Greater than or equal to,	
<= --> Less than or equal to	

Table 2: Comparison operator and Logical operator

3.Iterative Statements in Python

Iterative statements are used to execute a block of code repeatedly until a condition becomes false. Iteration is also known as looping. It saves time and memory and also makes programs efficient.

Python mainly provides two types of loops:

i) for loop:

The for loop is used to repeat a block of code a fixed number of times.

Syntax

```
for variable in sequence:
    statements
```

Example

```
for i in range(5):
    print(i)
```

ii) while loop:

The **while** loop executes code as long as the condition remains **true**.

Syntax

```
while condition:
    statements
```

Example

```
count = 1

while count <= 5:
    print(count)
    count += 1
```

4. Loop Control Statements

Loop control statements are used to control the execution of loops in Python. They help change the normal flow of a loop based on certain conditions.

Python provides three main loop control statements like **break**, **continue** and **pass**.

These statements are commonly used with **for loops** and **while loops**

1. break Statement

The break statement **stops** the loop immediately, when conditions are met .

Example

```
for i in range(10):
    if i == 5:
        break
    print(i)
```

2. continue Statement

The continue statement **skips** the current iteration and moves to the next iteration.

Example

```
for i in range(5):
    if i == 2:
        continue
    print(i)
```

3. pass Statement

The pass statement is a null statement. It is used when no action is required. It acts as a placeholder for future code.

Example

```
for i in range(5):
    if i == 2:
        pass
    print(i)
```

5. Random Module in Python

The random module is used to generate **random numbers** and random selections in Python programs. It is mainly used in Games, Simulations, Password generators, Lottery systems. Etc.

Importing Random Module

Before using the random module, it must be imported.

Syntax

```
import random
```

Function	Purpose
random.randint()	Generates random integer
random.choice()	Selects random item
random.random()	Generates random float number

Table 3: Common Functions of Random Module

Example 1: Random Integer

```
import random
number = random.randint(1, 10)
print(number)
```

Sample Output

7

Example 2: Random Choice

```
import random
game = ["Rock", "Paper", "Scissors"]
choice = random.choice(game)
print(choice)
```

Sample Output

Rock

6.print() statement

It is used to display output on the screen. It is one of the most commonly used functions in Python.

Syntax

```
print("message")
```

Printing	Statement	Output
Text	print("Welcome to Python")	Welcome to Python
Variables	name = "Rahul" print(name)	Rahul
Multiple Values	name = "John" age = 20 print("Name:", name, "Age:", age)	Name: John Age: 20
F-String print	name = "Ram" age = 25 print(f" Name :{name},Age: ,{age}")	Name: Ram Age: 25

Table 4: Examples of Print Statements

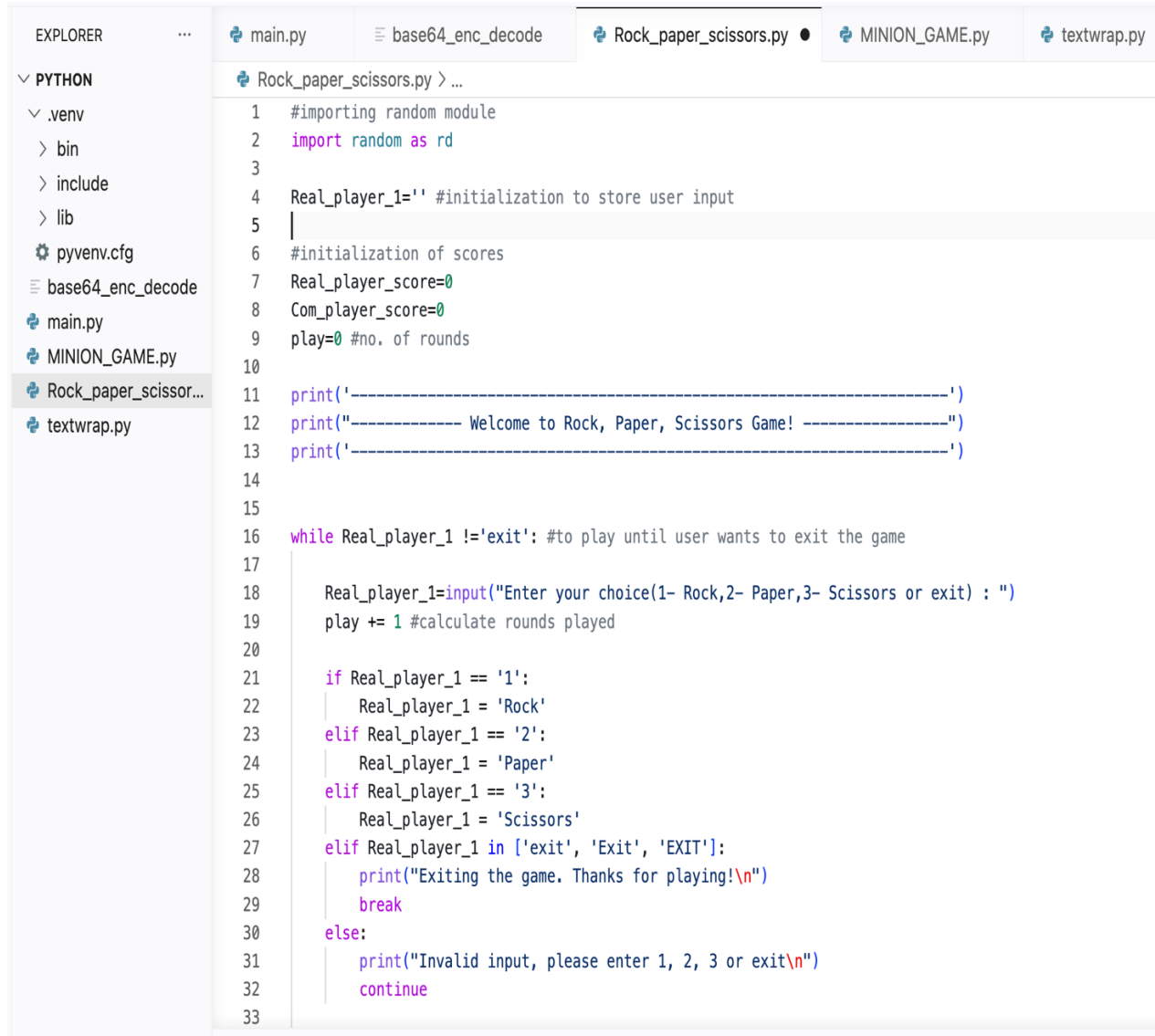
7.input() statement

This function is used to take user input from the keyboard. It always returns the entered value as a **string**, even if the user types numbers, so you often need to convert it into corresponding type using **typecasting**.

Usage	Statements	Output
String input	name = input("Name: ")	Stores text
Integer input	num = int(input("Number: "))	Stores number
Float input	val = float(input("Value: "))	Stores decimal
Multiple inputs	x, y = input().split()	Stores two values

Table 5: Examples of Input Statements

SOURCE CODE



```
1  #importing random module
2  import random as rd
3
4  Real_player_1='' #initialization to store user input
5  |
6  #initialization of scores
7  Real_player_score=0
8  Com_player_score=0
9  play=0 #no. of rounds
10
11 print('-----')
12 print("----- Welcome to Rock, Paper, Scissors Game! -----")
13 print('-----')
14
15
16 while Real_player_1 !='exit': #to play until user wants to exit the game
17
18     Real_player_1=input("Enter your choice(1- Rock,2- Paper,3- Scissors or exit) : ")
19     play += 1 #calculate rounds played
20
21     if Real_player_1 == '1':
22         Real_player_1 = 'Rock'
23     elif Real_player_1 == '2':
24         Real_player_1 = 'Paper'
25     elif Real_player_1 == '3':
26         Real_player_1 = 'Scissors'
27     elif Real_player_1 in ['exit', 'Exit', 'EXIT']:
28         print("Exiting the game. Thanks for playing!\n")
29         break
30     else:
31         print("Invalid input, please enter 1, 2, 3 or exit\n")
32         continue
33
```

```
▼ PYTHON
  ▼ .venv
    > bin
    > include
    > lib
    ⚙ pyenv.cfg
    ≡ base64_enc_decode
    📄 main.py
    📄 MINION_GAME.py
    📄 Rock_paper_scissor...
    📄 textwrap.py

Rock_paper_scissors.py > ...
33
34 #printing current playing Round, Real_palyer choice and Computer choice
35
36 print(f"\n----- Round {play} -----")#current playing round
37
38 print(f"Real player choice is : {Real_player_1}")#Real_player_1 choice
39
40 Com_player_1=rd.choice(['Rock', 'Paper', 'Scissors']) #computer random choice from options list
41
42 print(f"Comp random choice is : {Com_player_1}")#Comp_player choice
43
44
45 # Game logic:
46 if Real_player_1 == Com_player_1:
47     print("It's a tie!\n")
48
49     Real_player_score=Real_player_score
50     Com_player_score=Com_player_score
51
52 elif (Real_player_1 == 'Rock' and Com_player_1 == 'Scissors') \
53      or (Real_player_1 == 'Paper' and Com_player_1 == 'Rock') \
54      or (Real_player_1 == 'Scissors' and Com_player_1 == 'Paper'):
55
56     print("Real_player win!\n")
57
58     Real_player_score += 1
59 else:
60     print("Computer wins!\n")
61     Com_player_score += 1
62
```

```
▼ PYTHON
  ▼ .venv
    > bin
    > include
    > lib
    ⚙ pyenv.cfg
    ≡ base64_enc_decode
    📄 main.py
    📄 MINION_GAME.py
    📄 Rock_paper_scissor...
    📄 textwrap.py

Rock_paper_scissors.py > ...
62
63 # Print the final scores and declare the winner
64
65 print('----- Final Scores -----')
66 print(f"No.of rounds played : {play-1} ")
67 print('-----')
68 print(f"Real_player: {Real_player_score}")
69 print(f"Computer   : {Com_player_score}\n")
70
71
72 if Real_player_score > Com_player_score:
73     print("Congratulations! You won the game!\n")
74 elif Real_player_score < Com_player_score:
75     print("Computer wins the game! Better luck next time.\n")
76 else:
77     print("The game is a tie!\n")
78
79 print('-----')
80 |
```

EXPLANATION OF SOURCE CODE

The program begins by importing the `random` module as `rd`. This module is used to generate a random choice for the computer from the available options: **Rock, Paper, and Scissors**. Next, the following variables `Real_player_1`, `Real_player_score`, `Com_player_score` and `play` are initialized.

- `Real_player_1` stores the player's current choice.
- `Real_player_score` stores the player's score.
- `Com_player_score` stores the computer's score.
- `play` keeps track of the total number of rounds.

The program then prints a welcome message like **"Welcome to Rock, Paper, Scissors Game!"**

A `while` loop (Iterative statements) is used to keep the game running until the player enters `exit`. Inside the loop, conditional statements (`if-elif-else` statements) are used to (for convenience), the player is asked to enter a choice using `input()`. The entered value is checked using conditional statements:

- If the user input is `1`, it is converted to **Rock**.
- If the user input is `2`, it is converted to **Paper**.
- If the user input is `3`, it is converted to **Scissors**.
- If the user input is `exit`, the **game ends**.
- If the user input is **invalid**, an **error message is displayed** and the program continues to the next iteration.

After a valid choice is entered, the current round number (`play`) and the player's choice (`Real_player_1`) are displayed using `print` statement with `f-strings`.

The computer then randomly selects one option using `rd.choice()` using `random` module and it is displayed using printing (`Com_player_1`) variable.

The game result is determined using “**Game Logic**” conditional statements (**if-elif-else statements**):

- If both **player** and **computer choices** are the **same** like (Rock-Rock, Scissor-Scissor and Paper-Paper), the round is a **tie** and both **scores are same**.
- If the **real player's choice beats** the computer's choice according to the rules, the player wins and the player's **score is increased**.
- Otherwise, the computer **wins** and its **score is increased**.

When the player exits the game **using** `Real_player_1 in ['exit', 'Exit', 'EXIT']`, game ends. The final scores are displayed along with the total number of rounds played is `play-1` as last round is ends the game.

Finally, the program compares both scores and declares the overall winner:

- If the player's score is greater, a congratulatory message is shown using print as `print("Congratulations! You won the game!\n")`
- If the computer's score is greater, the program declares the computer as the winner and using print statements as `print("Computer wins the game! Better luck next time.\n")`
- If both scores are equal, the game is declared a tie and using print statements as `print("The game is a tie!\n")`

This project demonstrates the use of loops, user input, random values, variables, and conditional statements in Python. Thus, conditional statements are the main logic-building blocks of this game.

OUTPUT

Sample Output – Rounds (Real player, computer win and Tie scenario)

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
(.venv) [redacted] python % /[redacted].com/Desktop/python/
.venv/bin/python /Users/[redacted].com/Desktop/python/Rock_paper_scissors.py
----- Round 7 -----
Real player choice is : Rock
Comp random choice is : Paper
Computer wins!

Enter your choice(1- Rock,2- Paper,3- Scissors or exit) : 2

----- Round 8 -----
Real player choice is : Paper
Comp random choice is : Paper
It's a tie!

Enter your choice(1- Rock,2- Paper,3- Scissors or exit) : 3

----- Round 9 -----
Real player choice is : Scissors
Comp random choice is : Paper
Real_player win!

Enter your choice(1- Rock,2- Paper,3- Scissors or exit) : █
Ln 77, Col 4 (30 selected) Spaces: 4 UTF-8 LF {} Python 3.9.6
```

Sample Output – Rounds (Normal input cases)

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
(.venv) [redacted]@INSML-FJ7CG6NR2L python % /Users/[redacted].com/Desktop/python/
.venv/bin/python /Users/[redacted].com/Desktop/python/Rock_paper_scissors.py
----- Round 9 -----
Real player choice is : Scissors
Comp random choice is : Paper
Real_player win!

Enter your choice(1- Rock,2- Paper,3- Scissors or exit) : 1

----- Round 10 -----
Real player choice is : Rock
Comp random choice is : Rock
It's a tie!

Enter your choice(1- Rock,2- Paper,3- Scissors or exit) : 2

----- Round 11 -----
Real player choice is : Paper
Comp random choice is : Rock
Real_player win!

Enter your choice(1- Rock,2- Paper,3- Scissors or exit) : 3

----- Round 12 -----
Real player choice is : Scissors
Comp random choice is : Scissors
It's a tie!

Enter your choice(1- Rock,2- Paper,3- Scissors or exit) : █
```

Sample Output – Rounds (For Invalid input cases)

```
.venv/bin/python /Users/[redacted].com/Desktop/python/Rock_paper_scissors.py

----- Round 3 -----
Real player choice is : Scissors
Comp random choice is : Scissors
It's a tie!

Enter your choice(1- Rock,2- Paper,3- Scissors or exit) : 1

----- Round 4 -----
Real player choice is : Rock
Comp random choice is : Scissors
Real_player win!

Enter your choice(1- Rock,2- Paper,3- Scissors or exit) : 4
Invalid input, please enter 1, 2, 3 or exit

Enter your choice(1- Rock,2- Paper,3- Scissors or exit) : a
Invalid input, please enter 1, 2, 3 or exit

Enter your choice(1- Rock,2- Paper,3- Scissors or exit) : █
```

Ln 77, Col 4 (30 selected) Spaces: 4 UTF-8 LF {} Python 3.9.6

Sample Output – Rounds (For exit cases)

```
Enter your choice(1- Rock,2- Paper,3- Scissors or exit) : 3

----- Round 12 -----
Real player choice is : Scissors
Comp random choice is : Scissors
It's a tie!

Enter your choice(1- Rock,2- Paper,3- Scissors or exit) : exit
Exiting the game. Thanks for playing!
```

Final Output

```
----- Round 12 -----
Real player choice is : Scissors
Comp random choice is : Scissors
It's a tie!

Enter your choice(1- Rock,2- Paper,3- Scissors or exit) : exit
Exiting the game. Thanks for playing!

----- Final Scores -----
No.of rounds played : 12

Real_player: 4
Computer   : 2

Congratulations! You won the game!
```

> OUTLINE
> TIMELINE

(.venv) [redacted] python %

Ln 77, Col 4 (30 selected) Spaces: 4 UTF-8 LF {} Python 3.9.6

CONCLUSION

This mini project successfully demonstrates the implementation of a Rock, Paper, Scissors game using Python. The project is simple, interactive, and useful for understanding fundamental programming concepts. By developing this project, I learned how to use variables to store values, conditional statements to make decisions, loops to repeat actions, and the random module to generate computer choices.

The project also helped improve my logical thinking and problem-solving abilities. It shows how a real-world game can be converted into a computer program using basic programming techniques. User input validation and score tracking make the project more practical and interactive.

Overall, this project is a good example of beginner-level Python programming and serves as an excellent learning exercise. It builds a strong foundation for creating more advanced Python applications and games in the future.

In the future, this Rock-Paper-Scissors game can be enhanced by adding multiplayer functionality, improved graphics and animations, having splayer profiles, and an online leaderboard. Artificial intelligence can also be implemented to make the computer opponent more challenging. Additionally, the game can be developed as a mobile application or web application and include new game modes and achievements to provide a more engaging user experience.

REFERENCES

1. [OpenStax](#) – *Introduction to Python Programming* provided background on Python, variables, input/output, and beginner concepts. [[openstax.org](#)]
2. [CS50's Introduction to Programming with Python](#) – explained conditional statements, control flow, and `if`, `elif`, `else` usage in Python. [[cs50.harvard.edu](#)]
3. [GeeksforGeeks](#) – supported explanations about Python basics, variables, and data types. [[geeksforgeeks.org](#)]
4. [w3resource](#) – provided reference ideas for the Rock, Paper, Scissors project structure and sample outputs. [[w3resource.com](#)]
5. [Real Python](#) – gave conceptual guidance on creating a Rock, Paper, Scissors game in Python. [[realpython.com](#)]
6. "Python in Visual Studio Code". [code.visualstudio.com](#). Retrieved 1 December 2025.
7. "Project Jupyter". [Jupyter.org](#). Archived from the original on 12 October 2023. Retrieved 2 April 2025.
8. OpenAI. (2026) Microsoft. (2026). *Microsoft Copilot*. Retrieved May 30, 2026, from <https://copilot.microsoft.com> and *ChatGPT*.s from <https://chatgpt.com>